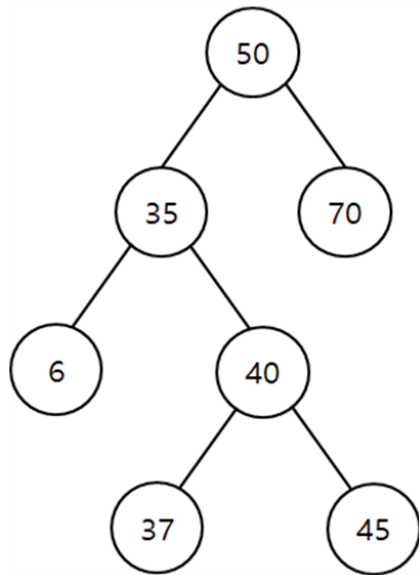


트리 응용

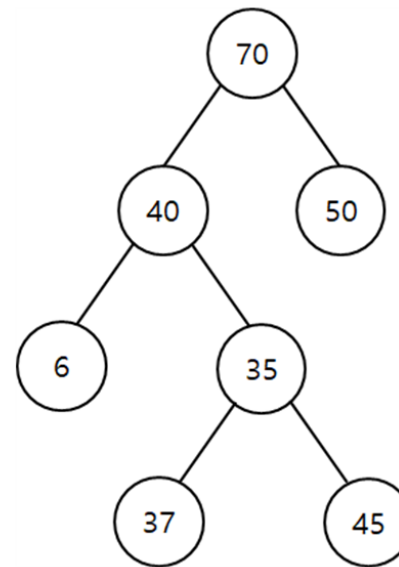
이진탐색트리의 개요

- 이진탐색트리란?
 - 데이터 검색에 매우 유용한 자료구조
 - 모든 노드는 유일한 키 값을 갖는다.
 - 한 노드의 키 값은 왼쪽 서브트리에서 모든 노드의 키 값들 보다 커야한다.
 - 한 노드의 키 값은 오른쪽 서브트리에서 모든 노드의 키 값들 보다 작아야 한다.
 - 각 서브트리에서 모든 노드의 키 값은 위의 조건을 충족해야 한다.

이진탐색트리의 개요



(그림 8.1) 이진탐색트리의 예



(그림 8.2) 이진트리의 예

가장 작은 값 -> 가장 왼쪽의 노드 값

이진탐색트리의 개요

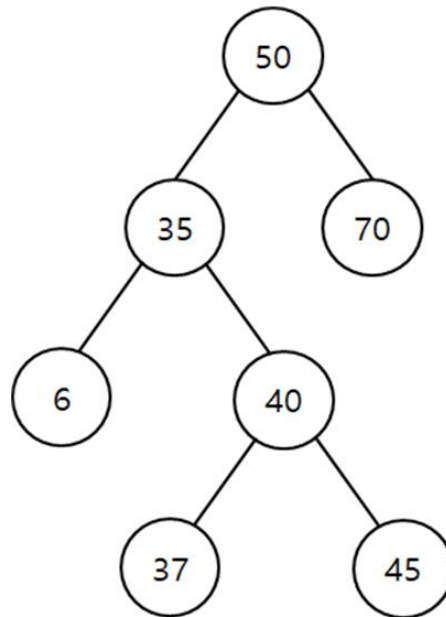
- 노드 구조

left_child	data	right_child
------------	------	-------------

(그림 8.3) 이진탐색트리의 노드 구조

```
Struct {  
    left_child  
    data  
    right_child } Bt_node
```

가장 작은값 찾기



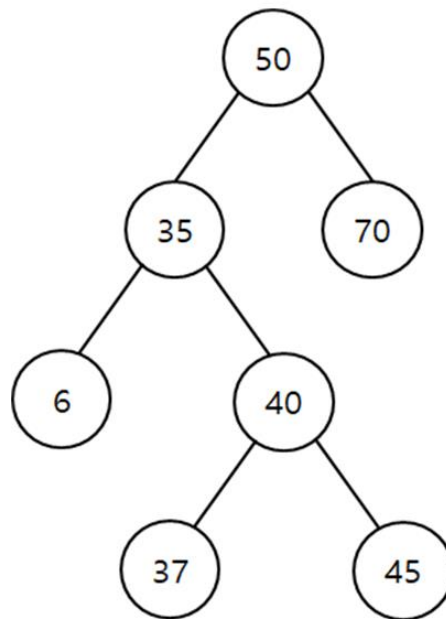
(그림 8.1) 이진탐색트리의 예

가장 작은값 찾기

1. Algorithm BST_min(root)
2. Input: 이진탐색트리의 루트 노드(root)
3. Output: 가장 작은 키 값
4. `temp <- root`
5. while (temp.left_child != Null) do
6. `temp <- temp.left_child`
7. return temp.data

수행시간 : $O(h)$

가장 큰값 찾기



가장 큰 자식노드값 -> 가장 오른쪽

(그림 8.1) 이진탐색트리의 예

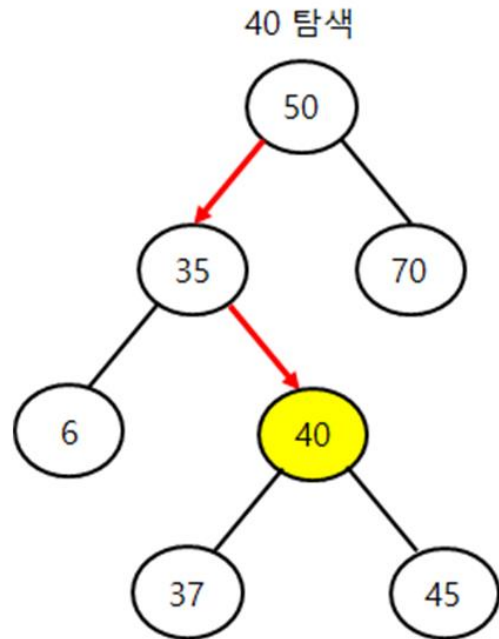
가장 큰값 찾기

1. Algorithm BST_max(root)
2. Input: 이진탐색트리의 루트 노드(root)
3. Output: 가장 큰 키 값
4. `temp <- root`
5. while (temp.right_child != Null) do
6. `temp <- temp.right_child`
7. return temp.data

수행시간 : $O(h)$

가장 큰 자식

특정 값 찾기



작은값 왼쪽 큰값 오른쪽

40은 50보다 작으니
왼쪽으로
40은 35보다 크니
오른쪽으로
찾았다

(그림 8.4) 이진탐색트리에서 탐색 예

특정 값 찾기

<재귀적 호출 방법>

1. Algorithm BST_search(root, key)
 2. Input: 이진탐색트리의 루트 노드(root), 탐색 키 값(key)
 3. Output: 키 값과 일치하는 노드
 4. if (root = Null) return Null
 5. if (key = root.data) return node
 6. if (key < root.data)
 7. return BST_search(root.left_child, key)
 8. else
 9. return BST_search(root.right_child, key)
 10. end
- 수행시간 : $O(h)$

특정 값 찾기

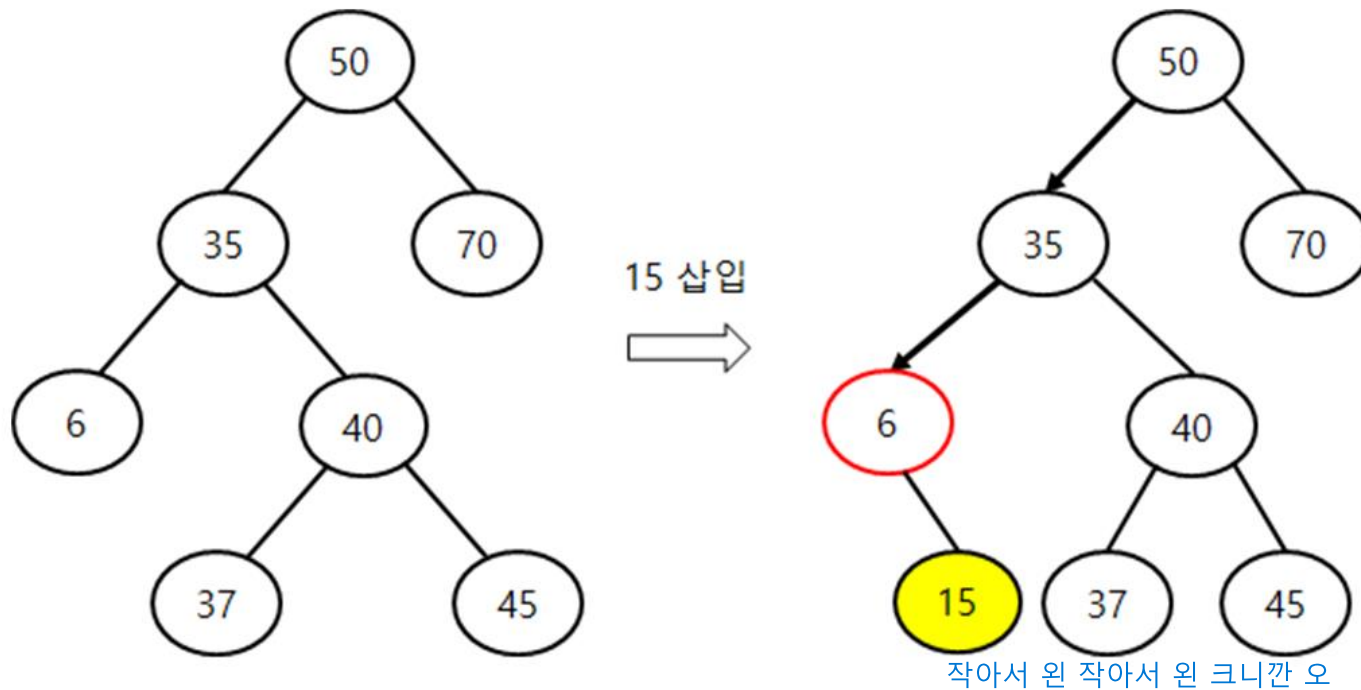
<반복적 방법>

1. Algorithm BST_search_it(root, key)
2. Input: 이진탐색트리의 루트 노드(root), 탐색 키 값(key)
3. Output: 키 값과 일치하는 노드
4. `node <- root`
5. `while (node != Null) do {`
6. `if (key = node.data) return node`
7. `if (key < node.data)`
8. `node <- node.left_child`
9. `else`
10. `node <- node.right_child`
11. `}`
12. `return Null`

수행시간 : $O(h)$

가장 이상적인 트리
크기가 중간인 값을 루트(최상단)으로
= 일직선 트리가 안됨

삽입 연산



(그림 8.5) 이진탐색트리에서 삽입 예

삽입 연산

1. Algorithm BST_insert(root, key)
2. Input: 이진탐색트리의 루트 노드(root), 삽입할 키 값(key)
3. Output: 새로운 키 값이 삽입된 이진탐색트리의 루트 노드
4. node $\leftarrow$ root
5. new_node $\leftarrow$ malloc(sizeof(node))
6. new_node.data $\leftarrow$ key
7. if (node = Null) {
8. root $\leftarrow$ new_node
9. return root }
10. while (node $\neq$ Null) do {
11. parent $\leftarrow$ node
12. if (key < node.data) node $\leftarrow$ node.left_child
13. else node $\leftarrow$ node.right_child }
14. if (key < parent.data) parent.left_child $\leftarrow$ new_node
15. else parent.right_child $\leftarrow$ new_node
16. return root

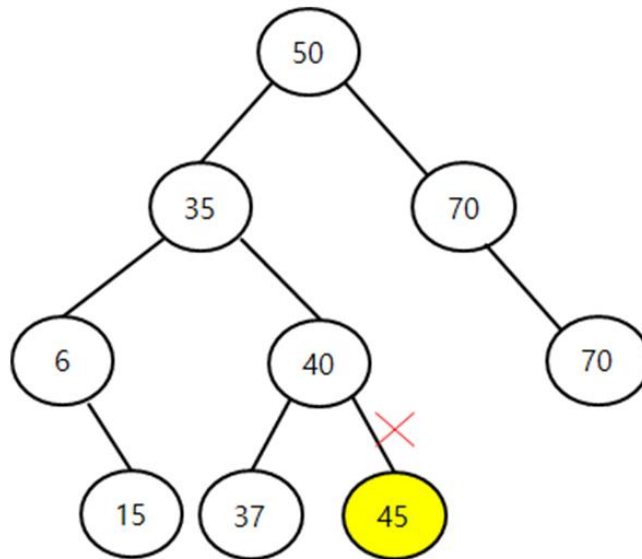
수행시간 : $O(h)$

삭제 연산

- (경우1) 삭제 노드가 단말 노드일 경우
- (경우2) 삭제 노드가 한 개의 자식 노드만을 가질 경우
- (경우3) 삭제 노드가 두 개의 자식 노드를 가질 경우

삭제연산

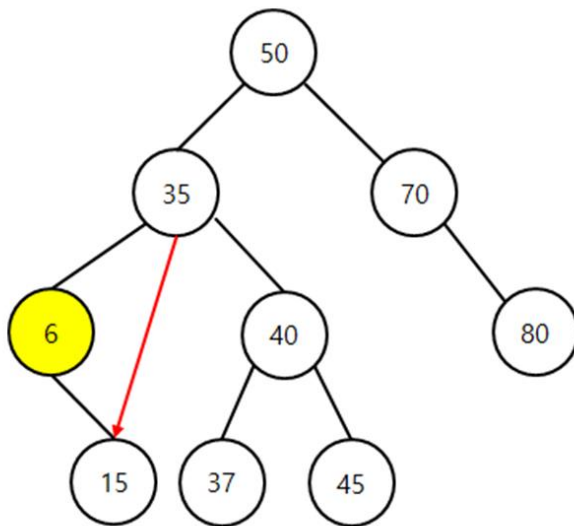
- (경우1) 삭제 노드가 단말 노드일 경우



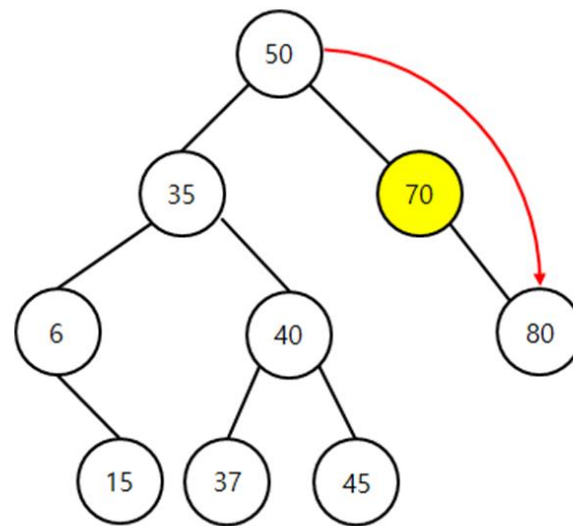
(그림 8.6) 키 값 45를 갖는 노드 삭제

삭제연산

- (경우2) 삭제 노드가 한 개의 자식 노드만을 가질 경우



(a) 키 값 6을 갖는 노드 삭제

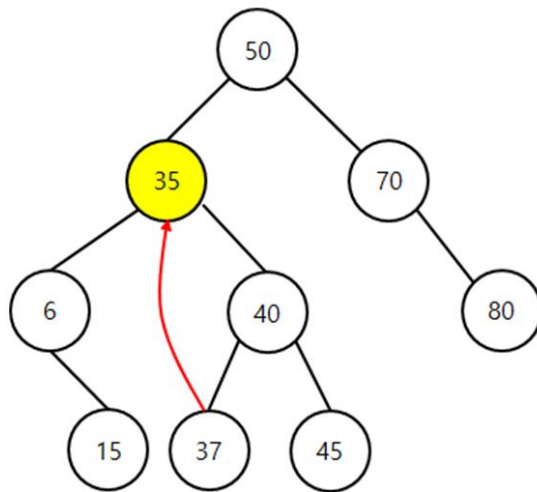


(b) 키 값 70을 갖는 노드 삭제

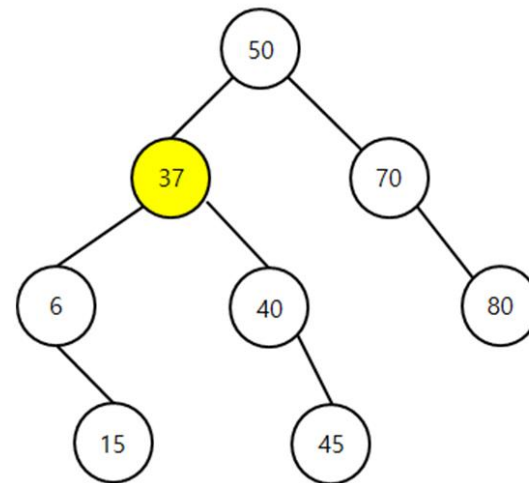
(그림 8.7) 한 개의 자식 노드를 갖는 키 값의 삭제 예

삭제연산

- (경우3) 삭제 노드가 두 개의 자식 노드를 가질 경우



(a) 키 값 35를 갖는 노드의 삭제

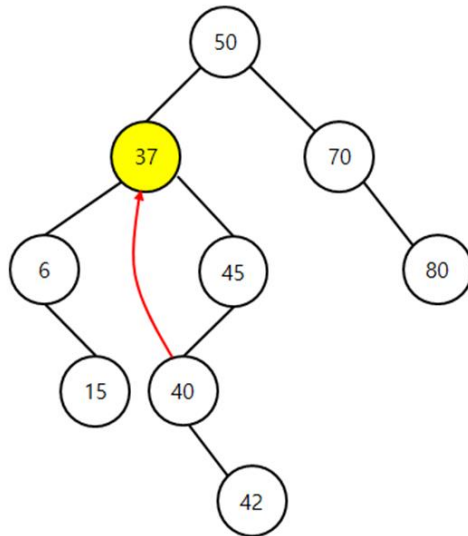


(b) 키 값 35 삭제 후 이진탐색트리

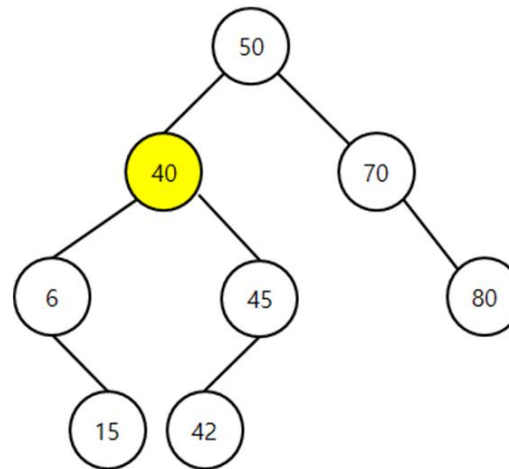
(그림 8.8) 두 개의 자식 노드를 갖는 키 값의 삭제 예

삭제연산

- (경우3) 삭제 노드가 두 개의 자식 노드를 가질 경우



(a) 키 값 37을 갖는 노드의 삭제



(b) 키 값 37을 삭제 후 이진탐색트리

(그림 8.9) 두 개의 자식 노드를 갖는 키 값의 삭제 예

삭제연산

1. Algorithm BST_delete(root, key)
2. Input: 이진탐색트리의 루트 노드(root), 삭제할 키 값(key)
3. Output: 삭제가 완료된 이진탐색트리의 루트 노드
4. curr <- root // 현재 노드
5. parent <- Null // 부모 노드
6. while (curr != Null and curr.data != key) do {
7. parent <- curr
8. if (key < curr.data) then
9. curr <- curr.left_child
10. else
11. curr <- curr.right_child
12. }
13. if (curr = Null) return error // 키 값이 발견되지 않음

삭제연산

```
// 경우1의 구현: 삭제 노드가 단말 노드
14. if ((curr.left_child = Null) and (curr.right_child = Null)) {
15.   if (curr != root) {
16.       if (parent.left_child = curr)
17.           parent.left_child <- Null
18.       else
19.           parent.right_child <- Null
20.   }
21.   else
22.       root <- Null
23.   free(curr) }
```

삭제연산

```
// 경우2의 구현: 삭제 노드가 한 개의 자식 노드를 갖는 경우
24. else if ((curr.left_child = Null) or (curr.right_child = Null)) {
25.     if (curr.left_child = Null)
26.         child <- curr.right_child
27.     else
28.         child <- curr.left_child
29.     if (curr != root) {
30.         if (curr = parent.left_child)
31.             parent.left_child <- child
32.         else
33.             parent.right_child <- child
34.     }
35.     else
36.         root <- child
37.     free(curr) }
```

삭제연산

```
// 경우3의 구현: 삭제 노드가 두 개의 자식 노드를 갖는 경우
38. else { // 오른쪽 서브트리에서 가장 작은 값을 찾음
39.     successor <- curr.right_child
40.     while (successor.left_child != Null) do
41.         successor <- successor.left_child
42.     k_value <- successor.data
43.     BST_delete(root, successor.data)
44.     curr.data <- k_value }
45. return root
```

알고리즘 분석

- 수행시간에 가장 크게 영향을 주는 부분은 6~12번 줄에서 키 값을 탐색하는 부분과 43번 줄에서 두 개의 자식 노드를 갖는 경우 삭제 위해 순환적으로 호출하는 부분이다.
- 키 값을 탐색하는 과정은 이진탐색트리에서 트리의 높이 h 에 비례한다. 또는 순환적으로 호출하는 부분 역시 트리의 높이 h 를 초과하지 않는다. 따라서 이 알고리즘의 수행시간은 $O(h)$ 가 된다.

논의

- 이진탐색트리는 데이터 탐색 분야의 응용에 아주 효율성이 좋은 자료구조이다.
- n 개의 데이터가 주어지면 순차 탐색의 경우 수행시간은 $O(n)$ 이 된다.
- 완전이진트리일 경우 수행시간은 $O(\log n)$ 이 된다.

균형 이진 탐색 트리

균형 이진 탐색 트리가 필요한 이유

- 일반적인 이진 탐색 트리(BST)는 삽입/삭제 순서에 따라 한쪽으로 치우친 형태가 될 수 있음
- 최악의 경우 선형구조 (ex. 정렬된 데이터를 그대로 삽입)
- 시간 복잡도 퇴화: $O(\log n) \rightarrow O(n)$
- 해결책: 트리의 균형을 유지하는 자료구조 도입

AVL 트리

정의

- 자기 균형 이진 탐색 트리
- 모든 노드에서 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이가 최대 1

균형 인수 (Balance Factor)

- 노드의 왼쪽 서브트리 높이 - 오른쪽 서브트리 높이
- 균형 인수가 $\{-1, 0, 1\}$ 이면 균형 상태
- 벗어나면 회전 연산으로 균형 회복

AVL 트리

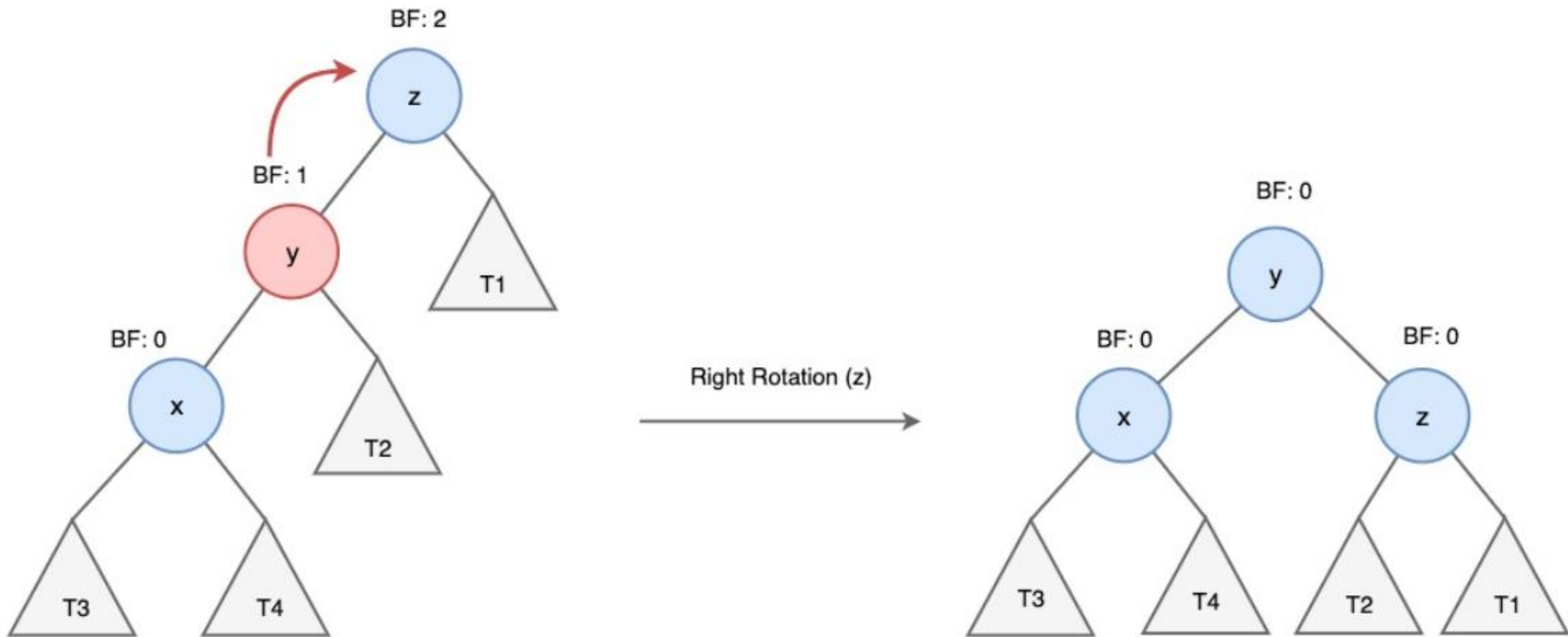
회전 종류

1. LL 회전 (단일 오른쪽 회전)
2. RR 회전 (단일 왼쪽 회전)
3. LR 회전 (왼쪽-오른쪽 회전)
4. RL 회전 (오른쪽-왼쪽 회전)

LL 회전 (Left-Left Rotation)

- 상황: 왼쪽 자식의 왼쪽 서브트리에 노드가 삽입되어 균형이 깨짐
- 대상 노드: 불균형이 처음 발생한 노드를 Z라 할 때, Z의 왼쪽 자식 Y, Y의 왼쪽 자식 X가 존재하는 경우
- 동작:
 - Z를 기준으로 오른쪽쪽으로 회전
 - Y가 새로운 루트로 올라오고, Z는 Y의 오른쪽 자식이 됨
 - X는 그대로 Y의 왼쪽 자식으로 유지됨

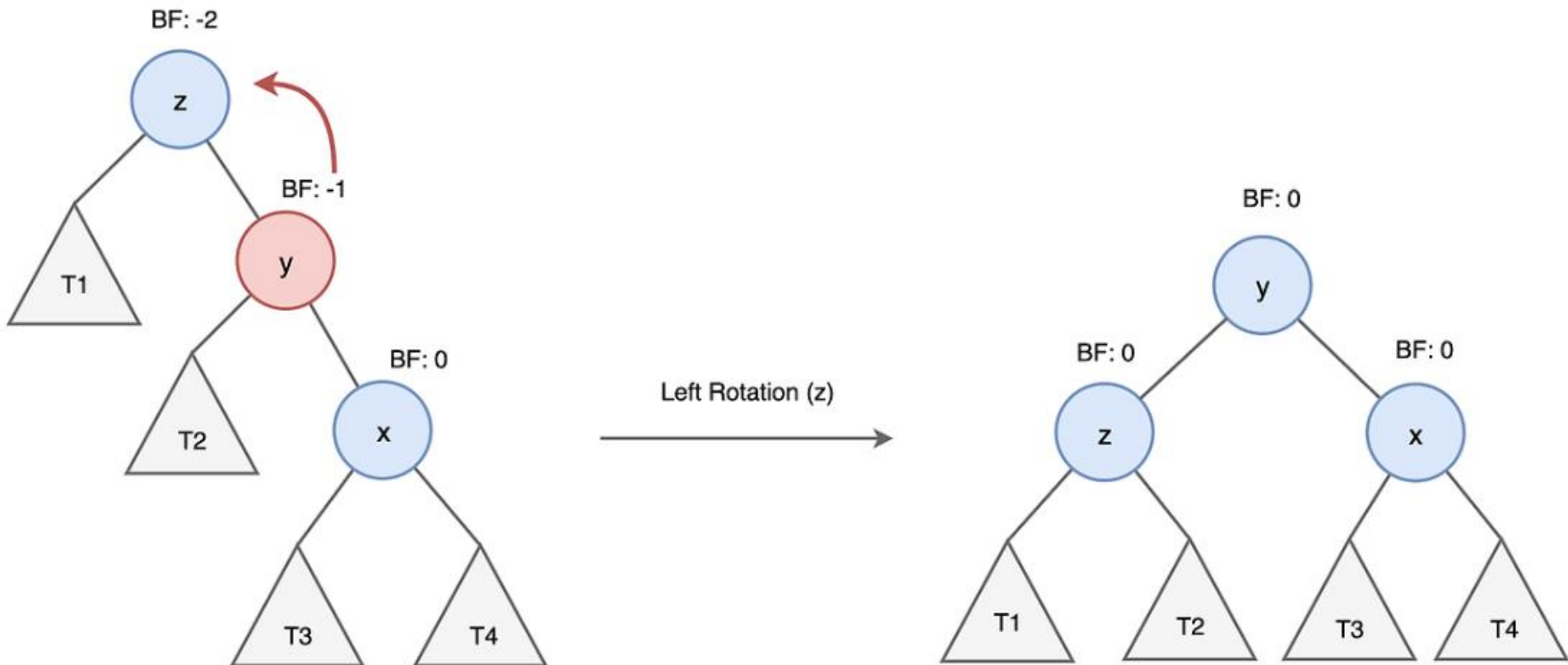
LL 회전 (Left-Left Rotation)



RR 회전 (Right-Right Rotation)

- **상황:** 오른쪽 자식의 오른쪽 서브트리에 노드가 삽입되어 균형이 깨짐
- **대상 노드:** 불균형이 처음 발생한 노드를 Z라 할 때, Z의 오른쪽 자식 Y, Y의 오른쪽 자식 X가 존재하는 경우
- **동작:**
 - Z를 기준으로 왼쪽으로 회전
 - Y가 새로운 루트로 올라오고, Z는 Y의 왼쪽 자식이 됨
 - X는 그대로 Y의 오른쪽 자식으로 유지됨

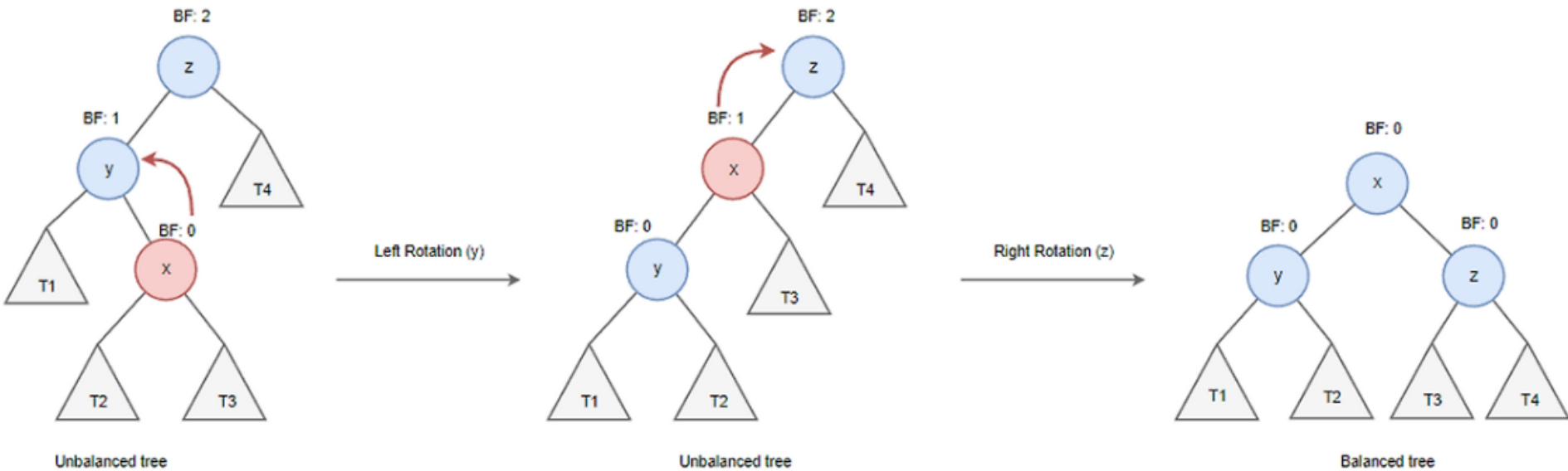
RR 회전 (Right-Right Rotation)



LR 회전 (Left-Right Rotation)

- **상황:** 왼쪽 자식의 오른쪽 서브트리에 노드가 삽입되어 균형이 깨짐
- **대상 노드:** 불균형이 처음 발생한 노드를 z 라 할 때, z 의 왼쪽 자식 y , y 의 오른쪽 자식 x 가 존재하는 경우
- **동작:**
 - y 를 기준으로 왼쪽으로 회전하여 x 가 y 의 부모가 되도록 함
 - z 를 기준으로 오른쪽으로 회전하여 x 가 z 의 부모가 되도록 함

LR 회전 (Left-Right Rotation)



RL 회전 (Right-Left Rotation)

- **상황:** 오른쪽 자식의 왼쪽 서브트리에 노드가 삽입되어 균형이 깨짐
- **대상 노드:** 불균형이 처음 발생한 노드를 Z라 할 때, Z의 오른쪽 자식 Y, Y의 왼쪽 자식 X가 존재하는 경우
- **동작:**
 - Y를 기준으로 오른쪽으로 회전하여 X가 Y의 부모가 되도록 함
 - Z를 기준으로 왼쪽으로 회전하여 X가 Z의 부모가 되도록 함

RL 회전 (Right-Left Rotation)

